

CONTENTS

Problem Statement.....	2
Dataset Description.....	2
How to obtain the dataset.....	3
Summary of Data Analysis.....	3
Map Reduce Analysis	5
1) Flight Count by Airline	5
2) Departures by Origin Airport.....	6
3) Flight Count by Month	8
4) Cancelled Flights by Airline	9
5) Average Arrival Delay by Airline.....	11
Pig Flight Data Analysis	12
1) Airline Flight Totals.....	12
2) Monthly Flight Totals	13
3) Top Origin Airports	14
4) Top Routes	15
5) Average Departure Delay by Airline	16
6) Cancellations by Airline	17
7) Average Arrival Delay by Month	18
MapReduce and Pig Comparison.....	19
Results and Findings.....	20
Conclusion	20

Problem Statement

Analyze the supplied 2015 U.S. flight dataset using Hadoop MapReduce and Apache Pig. The analysis identifies flight volumes, airport activity, monthly demand, cancellation patterns, and delay performance using the fields available in the dataset..

Dataset Description

Dataset: <https://www.kaggle.com/datasets/usdot/flight-delays>

The dataset contains three CSV files: flights.csv, airlines.csv, and airports.csv.

The main file, flights.csv, contains **5,819,079 flight records** with **31 columns**. It includes information such as flight date, airline code, flight number, origin airport, destination airport, departure delay, arrival delay, cancellation status, cancellation reason, distance, and air time.

The airlines.csv file contains airline codes and airline names. The airports.csv file contains airport codes, airport names, cities, states, countries, latitude, and longitude. These two files are used as lookup datasets to understand airline and airport codes in the main flight data.

Column Name	Description
YEAR	Year of the flight
MONTH	Month of the flight
DAY	Day of the month
DAY_OF_WEEK	Day of the week
AIRLINE	Airline code
FLIGHT_NUMBER	Flight number
TAIL_NUMBER	Aircraft tail number
ORIGIN_AIRPORT	Departure airport code
DESTINATION_AIRPORT	Arrival airport code
SCHEDULED_DEPARTURE	Scheduled departure time
DEPARTURE_TIME	Actual departure time
DEPARTURE_DELAY	Departure delay in minutes
TAXI_OUT	Time spent taxiing before takeoff
WHEELS_OFF	Actual wheels-off time
SCHEDULED_TIME	Scheduled flight duration

ELAPSED_TIME	Actual elapsed flight time
AIR_TIME	Time spent in the air
DISTANCE	Distance between origin and destination
WHEELS_ON	Actual wheels-on time
TAXI_IN	Time spent taxiing after landing
SCHEDULED_ARRIVAL	Scheduled arrival time
ARRIVAL_TIME	Actual arrival time
ARRIVAL_DELAY	Arrival delay in minutes
DIVERTED	Indicates whether the flight was diverted
CANCELLED	Indicates whether the flight was cancelled
CANCELLATION_REASON	Reason for flight cancellation
AIR_SYSTEM_DELAY	Delay caused by the air transportation system
SECURITY_DELAY	Delay caused by security-related issues
AIRLINE_DELAY	Delay caused by the airline
LATE_AIRCRAFT_DELAY	Delay caused by a late-arriving aircraft
WEATHER_DELAY	Delay caused by weather conditions

How to obtain the dataset

- Download the 2015 Flight Delays and Cancellations dataset from Kaggle.
- Extract flights.csv, airlines.csv, and airports.csv into the project data folder.
- The files are comma-separated CSV files; flights.csv is the primary analysis input.
- Run the MapReduce and Pig commands from Windows Administrator Command Prompt.

Summary of Data Analysis

This project analyzes the airline flight dataset using Hadoop MapReduce and Apache Pig. The purpose of the analysis is to identify airline activity, busy airports, monthly flight trends, cancellation patterns, and flight-delay performance from a large collection of flight records.

Hadoop MapReduce is used to process the dataset through Driver, Mapper, and Reducer classes. The Mapper reads each flight record and extracts the required fields. Hadoop groups records with the same key, and the Reducer combines these values to generate the final results. Apache

Pig is used for high-level analysis through Pig Latin scripts, which make it easier to load, filter, group, sort, and store the data.

- **Filtering:** Header records, invalid records, and incomplete flight records are excluded before analysis.
- **Data Organization:** Airline code, origin airport, month, and route fields are used as grouping keys to organize related flight records.
- **Summarization:** Reducers calculate total flight counts, airport departures, cancellation counts, and average delay values.
- **Performance Analysis:** Departure and arrival delay fields are analyzed to compare airline performance and identify airlines with higher delays.

MapReduce analyses performed:

- **Flight count by airline:** Counts the total number of flights operated by each airline.
- **Departures by origin airport:** Counts the number of flights departing from each airport.
- **Flight count by month:** Calculates the total number of flights for every month.
- **Cancelled flights by airline:** Counts cancelled flights for each airline.
- **Average arrival delay by airline:** Calculates the average arrival delay in minutes for each airline.

Pig analyses performed:

- **Airline flight totals:** Shows total flights for each airline with airline names.
- **Monthly flight totals:** Shows the total number of flights in each month.
- **Top origin airports:** Identifies the ten airports with the highest number of departures.
- **Top routes:** Identifies the ten busiest flight routes based on origin and destination airports.
- **Average departure delay by airline:** Calculates the average departure delay for each airline.
- **Cancellations by airline:** Shows the total number of cancelled flights for each airline.
- **Average arrival delay by month:** Calculates the average arrival delay for each month.

Map Reduce Analysis

1) Flight Count by Airline

Objective

The purpose of this analysis is to determine how many flight records were operated by each airline during 2015. The airline code is used as the grouping key.

Code

```
package flightanalysis;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Counts all scheduled flight records for each airline code. */
public class AirlineFlightCount {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private static final IntWritable ONE = new IntWritable(1);
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] f = value.toString().split(",", -1);
            if (f.length == 31 && !"YEAR".equals(f[0]) && !f[4].trim().isEmpty()) context.write(new Text(f[4]), ONE);
        }
    }
    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
            int total = 0; for (IntWritable value : values) total += value.get();
            context.write(key, new IntWritable(total));
        }
    }
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration(); conf.set("fs.defaultFS", "file:///"); conf.set("mapreduce.framework.name", "local");
        Job job = Job.getInstance(conf, "Flight count by airline");
        job.setJarByClass(AirlineFlightCount.class); job.setMapperClass(Map.class); job.setReducerClass(Reduce.class);
        job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Command to run

```
hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AirlineFlightCount data\flights.csv MapReduce\output\01_airline_flight_count
```

```
C:\Windows\System32>cd /d E:\BDAL\Final_Assignment\Flight_Data_Analysis

E:\BDAL\Final_Assignment\Flight_Data_Analysis>hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AirlineFlightCount data\flights.csv MapReduce\output\01_airline_flight_count
Map input records=5819080
Map output records=5819079
Reduce output records=14
map 100% reduce 100%
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/MapReduce/01_airline_flight_count/part-r-00000
AA      725984
AS      172521
B6      267048
DL      875881
EV      571977
F9      90836
HA      76272
MQ      294632
NK      117379
OO      588353
UA      515723
US      198715
VX      61903
WN      1261855
```

Selected execution output

Key	Result
WN	1,261,855
DL	875,881
AA	725,984
OO	588,353
EV	571,977

Interpretation

The flight count analysis provides an overview of airline activity in the dataset. Airlines with larger flight totals operated a greater number of recorded flights during the year. This result can be useful for understanding the relative operational volume of different airlines.

2) Departures by Origin Airport

Objective

This analysis determines the number of flights departing from each origin airport.

Code

```
package flightanalysis;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Counts departing flights at each origin airport. */
public class AirportDepartureCount {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private static final IntWritable ONE = new IntWritable(1);
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] f = value.toString().split(",", -1);
            if (f.length == 31 && !"YEAR".equals(f[0]) && !f[7].trim().isEmpty()) context.write(new Text(f[7]), ONE);
        }
    }

    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
```

```

        int total = 0; for (IntWritable value : values) total += value.get(); context.write(key, new IntWritable(total));
    }

}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration(); conf.set("fs.defaultFS", "file:///"); conf.set("mapreduce.framework.name", "local");
    Job job = Job.getInstance(conf, "Departures by origin airport");
    job.setJarByClass(AirportDepartureCount.class); job.setMapperClass(Map.class); job.setReducerClass(Reduce.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Command to run

```

hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AirportDepartureCount data\flights.csv
MapReduce\output\02_airport_departure_count

```

```

C:\Windows\System32>cd /d E:\BDAL\Final_Assignment\Flight_Data_Analysis

E:\BDAL\Final_Assignment\Flight_Data_Analysis>hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AirportDepartureCount data\flights.csv MapReduce\output\02_airport_departure_count
Map input records=5819080
Reduce output records=628
map 100% reduce 100%
Job completed successfully

```

```

E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/MapReduce/02_airport_departure_count/part-r-00000 | powershell -Command "$input | Select-Object -First 15"
10135 239
10136 201
10140 1816
10141 71
10146 87
10154 28
10155 148
10157 122
10158 248
10165 9
10170 31
10185 282
10208 223
10257 734
10268 57

```

Selected execution output

Key	Result
ATL	346,836
ORD	285,884
DFW	239,551
DEN	196,055
LAX	194,673

Interpretation

The result identifies the airports with the highest flight activity. A higher departure count indicates that the airport handled a larger number of departing flight records during the year. This information can be useful for identifying major or highly active airports in the dataset

3) Flight Count by Month

Objective

The purpose of this analysis is to determine the number of recorded flights in each month of 2015.

Code

```
package flightanalysis;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Counts flight records in each calendar month. */
public class MonthlyFlightCount {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private static final IntWritable ONE = new IntWritable(1);
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] f = value.toString().split(",", -1);
            if (f.length == 31 && !"YEAR".equals(f[0]) && !f[1].trim().isEmpty()) context.write(new Text(f[1]), ONE);
        }
    }
    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
            int total = 0; for (IntWritable value : values) total += value.get(); context.write(key, new IntWritable(total));
        }
    }
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration(); conf.set("fs.defaultFS", "file:///"); conf.set("mapreduce.framework.name", "local");
        Job job = Job.getInstance(conf, "Flight count by month");
        job.setJarByClass(MonthlyFlightCount.class); job.setMapperClass(Map.class); job.setReducerClass(Reduce.class);
        job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Command to run

```
hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.MonthlyFlightCount data\flights.csv MapReduce\output\03_monthly_flight_count
```

```
E:\B0AL\Final_Assignment\Flight_Data_Analysis>hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.MonthlyFlightCount data\flights.csv MapReduce\output\03_monthly_flight_count
Map input records=5819080
Reduce output records=12
map 100% reduce 100%
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/MapReduce/03_monthly_flight_count/part-r-00000
1      469968
10     486165
11     467972
12     479230
2      429191
3      504312
4      485151
5      496993
6      503897
7      520718
8      510536
9      464946
```

Selected execution output

Key	Result
January	469,968
March	504,312
July	520,718
August	510,536
December	479,230

Interpretation

Monthly flight counts show how flight activity changes throughout the year. The results indicate that the number of flights was not exactly the same in every month. This type of analysis can help identify periods of higher and lower flight activity.

4) Cancelled Flights by Airline

Objective

This analysis identifies the number of cancelled flights associated with each airline.

Code

```
package flightanalysis;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Counts cancelled flights for each airline. */
public class CancellationByAirline {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private static final IntWritable ONE = new IntWritable(1);
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] f = value.toString().split(",", -1);
            if (f.length == 31 && !"YEAR".equals(f[0]) && "1".equals(f[24]) && !f[4].trim().isEmpty()) context.write(new Text(f[4]), ONE);
        }
    }

    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
```

```

public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
    int total = 0; for (IntWritable value : values) total += value.get(); context.write(key, new IntWritable(total));
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration(); conf.set("fs.defaultFS", "file:///"); conf.set("mapreduce.framework.name", "local");
    Job job = Job.getInstance(conf, "Cancelled flights by airline");
    job.setJarByClass(CancellationByAirline.class); job.setMapperClass(Map.class); job.setReducerClass(Reduce.class);
    job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Command to run

```

hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.CancellationByAirline data\flights.csv
MapReduce\output\04_cancellation_by_airline

```

```

E:\BDAL\Final_Assignment\Flight_Data_Analysis>hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.CancellationByAirline data\flights.csv MapReduce\output\04_cancellation_by_airline
Map input records=5819080
Map output records=89884
Reduce output records=14
Job completed successfully

```

```

E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/MapReduce/04_cancellation_by_airline/part-r-00000
AA      10919
AS       669
B6     4276
DL     3824
EV     15231
F9       588
HA       171
MQ     15025
NK     2004
OO     9960
UA     6573
US     4067
VX       534
WN     16043

```

Selected execution output

Key	Result
WN	16,043
EV	15,231
MQ	15,025
AA	10,919
OO	9,960

Interpretation

This analysis shows which airlines have the highest number of cancelled flight records. However, the cancellation count alone does not represent a cancellation rate because airlines operate different numbers of flights. Therefore, the result should be considered as an absolute cancellation count.

5) Average Arrival Delay by Airline

Objective

The purpose of this analysis is to calculate the average recorded arrival delay for each airline.

Code

```
package flightanalysis;

import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Calculates mean recorded arrival delay, in minutes, for each airline. */
public class AverageArrivalDelayByAirline {
    public static class Map extends Mapper<LongWritable, Text, Text, Text> {
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String[] f = value.toString().split(",", -1);
            if (f.length == 31 && !"YEAR".equals(f[0]) && !f[4].trim().isEmpty() && !f[22].trim().isEmpty()) {
                try { context.write(new Text(f[4]), new Text(f[22] + "\t1")); } catch (NumberFormatException ignored) { }
            }
        }
    }
    public static class Reduce extends Reducer<Text, Text, Text, Text> {
        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
            double sum = 0; long count = 0;
            for (Text value : values) { String[] pair = value.toString().split("\t"); sum += Double.parseDouble(pair[0]); count += Long.parseLong(pair[1]); }
            context.write(key, new Text(String.format("%.2f", (double) sum / count)));
        }
    }
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration(); conf.set("fs.defaultFS", "file:///"); conf.set("mapreduce.framework.name", "local");
        Job job = Job.getInstance(conf, "Average arrival delay by airline");
        job.setJarByClass(AverageArrivalDelayByAirline.class); job.setMapperClass(Map.class); job.setReducerClass(Reduce.class);
        job.setMapOutputKeyClass(Text.class); job.setMapOutputValueClass(Text.class); job.setOutputKeyClass(Text.class); job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0])); FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Command to run

```
hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AverageArrivalDelayByAirline data\flights.csv
MapReduce\output\05_average_arrival_delay_by_airline
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hadoop jar MapReduce\build\flightanalysis.jar flightanalysis.AverageArrivalDelayByAirline data\flights.csv MapReduce\output\05_average_arrival_delay_by_airline

Map input records=5819080
Map output records=5714088
Reduce output records=14
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hd fs -cat /Final_Project/MapReduce/05_average_arrival_delay_by_airline/part-r-00000
AA      3.45
AS     -0.98
B6      6.68
DL      0.19
EV      6.59
F9     12.50
HA      2.02
MQ      6.46
NK     14.47
OO      5.85
UA      5.43
US      3.71
VX      4.74
WN      4.37
```

Selected execution output

Key	Result
NK	14.47
F9	12.50
B6	6.68
EV	6.59
MQ	6.46

Interpretation

Average arrival delay provides a way to compare the recorded delay performance of airlines. A higher average value indicates that flights from that airline experienced greater average arrival delays among the records included in the calculation.

Pig Flight Data Analysis

1) Airline Flight Totals

Objective

The purpose of this analysis is to calculate the total number of flights operated by each airline and display the corresponding airline names.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departur
e:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double
,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellati
on_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);

airlines = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\airlines.csv' USING PigStorage(',') AS (code:chararray,airline_name:chararray);
valid_flights = FILTER flights BY year IS NOT NULL;
valid_airlines = FILTER airlines BY code != 'IATA_CODE';
grouped = GROUP valid_flights BY airline;
totals = FOREACH grouped GENERATE group AS airline_code, COUNT(valid_flights) AS total_flights;
named = JOIN totals BY airline_code, valid_airlines BY code;

result = FOREACH named GENERATE totals::airline_code AS airline_code, valid_airlines::airline_name AS airline_name, totals::total_flights AS
total_flights;
ordered = ORDER result BY total_flights DESC;

STORE ordered INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis/output/01_airline_flight_totals' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis/scripts/01_airline_flight_totals.pig
```

```
Microsoft Windows [Version 10.0.26100.0]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd /d E:\BDAL\Final_Assignment\Flight_Data_Analysis

E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis/scripts/01_airline_flight_totals.pig
Pig features: HASH_JOIN, GROUP_BY, ORDER_BY, FILTER
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/01_airline_flight_totals/part-r-00000
WN|Southwest Airlines Co.|1261855
DL|Delta Air Lines Inc.|875881
AA|American Airlines Inc.|725984
OO|Skywest Airlines Inc.|588353
EV|Atlantic Southeast Airlines|571977
UA|United Air Lines Inc.|515723
MQ|American Eagle Airlines Inc.|294632
B6|JetBlue Airways|267048
US|US Airways Inc.|198715
AS|Alaska Airlines Inc.|172521
NK|Spirit Air Lines|117379
F9|Frontier Airlines Inc.|90836
HA|Hawaiian Airlines Inc.|76272
VX|Virgin America|61903
```

Interpretation

This analysis provides a more understandable representation of airline activity because it combines airline codes with airline names. It also demonstrates how Pig can perform grouping and joining operations on related datasets.

2) Monthly Flight Totals

Objective

The purpose of this analysis is to determine the total number of flights recorded during each month.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departur
e:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double
,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellati
on_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);

valid = FILTER flights BY year IS NOT NULL;
grouped = GROUP valid BY month;
result = FOREACH grouped GENERATE group AS month, COUNT(valid) AS total_flights;
ordered = ORDER result BY month ASC;

STORE ordered INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis/output/02_monthly_flight_totals' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis\scripts\02_monthly_flight_totals.pig
```

```
C:\Windows\System32>cd /d E:\BDAL\Final_Assignment\Flight_Data_Analysis
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\02_monthly_flight_totals.pig
```

```
Pig features: GROUP_BY, ORDER_BY, FILTER
```

```
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/02_monthly_flight_totals/part-r-000000
1|469968
2|429191
3|504312
4|485151
5|496993
6|503897
7|520718
8|510536
9|464946
10|486165
11|467972
12|479230
```

Interpretation

The monthly totals provide an overview of how flight activity changes across the year. This allows months with relatively higher or lower numbers of recorded flights to be identified.

3) Top Origin Airports

Objective

The purpose of this analysis is to identify the ten airports with the highest number of departing flights.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departur
e:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double
,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellati
on_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);
```

```
valid = FILTER flights BY year IS NOT NULL AND origin IS NOT NULL;
grouped = GROUP valid BY origin;
counts = FOREACH grouped GENERATE group AS origin_airport, COUNT(valid) AS departures;
ordered = ORDER counts BY departures DESC;
```

```
result = LIMIT ordered 10;
```

```
STORE result INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis\output\03_top_origin_airports' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis\scripts\03_top_origin_airports.pig
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\03_top_origin_airports.pig

Pig features: GROUP_BY, ORDER_BY, LIMIT, FILTER

Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/03_top_origin_airports/part-r-00000
ATL|346836
ORD|285884
DFW|239551
DEN|196055
LAX|194673
SFO|148008
PHX|146815
IAH|146622
LAS|133181
MSP|112117
```

Interpretation

This analysis identifies the busiest origin airports according to the number of recorded departures. It provides a simple way to determine which airports had the greatest level of departure activity in the dataset.

4) Top Routes

Objective

The purpose of this analysis is to identify the busiest flight routes based on origin and destination airports.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departur
e:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double
,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellati
on_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);
```

```
valid = FILTER flights BY year IS NOT NULL AND origin IS NOT NULL AND destination IS NOT NULL;
grouped = GROUP valid BY (origin, destination);
```

```
counts = FOREACH grouped GENERATE FLATTEN(group) AS (origin_airport, destination_airport), COUNT(valid) AS total_flights;
ordered = ORDER counts BY total_flights DESC;
```

```
result = LIMIT ordered 10;
```

```
STORE result INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis\output\04_top_routes' USING PigStorage("");
```

Command to run

```
pig -x local -f PigAnalysis\scripts\04_top_routes.pig
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\04_top_routes.pig
Pig features: GROUP_BY, FOREACH, FLATTEN, ORDER_BY, LIMIT
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/04_top_routes/part-r-00000
SFO|LAX|13744
LAX|SFO|13457
JFK|LAX|12016
LAX|JFK|12015
LAS|LAX|9715
LGA|ORD|9639
LAX|LAS|9594
ORD|LGA|9575
SFO|JFK|8440
JFK|SFO|8437
```

Interpretation

The top-route analysis shows which airport pairs had the highest number of recorded flights. It is useful for identifying heavily used connections within the flight network.

5) Average Departure Delay by Airline

Objective

This analysis calculates the average departure delay for each airline.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departure:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellation_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);

valid = FILTER flights BY year IS NOT NULL AND departure_delay IS NOT NULL;
grouped = GROUP valid BY airline;
result = FOREACH grouped GENERATE group AS airline_code, AVG(valid.departure_delay) AS average_departure_delay_minutes;
ordered = ORDER result BY average_departure_delay_minutes DESC;

STORE ordered INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis\output\05_average_departure_delay_by_airline' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis\scripts\05_average_departure_delay_by_airline.pig
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\05_average_departure_delay_by_airline.pig
Pig features: GROUP_BY, AVG, ORDER_BY, FILTER
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/05_average_departure_delay_by_airline/part-r-00000
NK|15.944765880783688
UA|14.435441010805953
F9|13.350858345331709
B6|11.5143526744102
WN|10.581986295158847
MQ|10.125188203309524
VX|9.022595096521952
AA|8.908856346719806
EV|8.71593449776958
OO|7.801103808415331
DL|7.36925417661782
US|6.141136917746696
AS|1.7858007096736666
HA|0.48571315965790407
```

Interpretation

Average departure delay provides an airline-level view of departure performance. Airlines with larger average values experienced greater recorded departure delays among the valid records used in the calculation.

The report identifies **NK** as having the highest average departure delay at approximately **15.94 minutes**.

6) Cancellations by Airline

Objective

The purpose of this analysis is to calculate the total number of cancelled flights for each airline.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departure:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellation_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);

cancelled_flights = FILTER flights BY year IS NOT NULL AND cancelled == 1;
grouped = GROUP cancelled_flights BY airline;
result = FOREACH grouped GENERATE group AS airline_code, COUNT(cancelled_flights) AS cancelled_flights;
ordered = ORDER result BY cancelled_flights DESC;

STORE ordered INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis\output\06_cancellations_by_airline' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis\scripts\06_cancellations_by_airline.pig
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\06_cancellations_by_airline.pig
Pig features: FILTER, GROUP_BY, ORDER_BY
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/06_cancellations_by_airline/part-r-00000
WN|16043
EV|15231
MQ|15025
AA|10919
OO|9960
UA|6573
B6|4276
US|4067
DL|3824
NK|2004
AS|669
F9|588
VX|534
HA|171
```

Interpretation

This analysis provides an airline-wise comparison of cancellation counts. It helps identify airlines with larger numbers of cancelled flight records in the dataset.

As with the MapReduce cancellation analysis, these are absolute counts and do not represent cancellation percentages.

7) Average Arrival Delay by Month

Objective

The purpose of this analysis is to calculate the average arrival delay for each month of 2015.

Script

```
flights = LOAD 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\data\flights.csv' USING PigStorage(',') AS
(year:int,month:int,day:int,day_of_week:int,airline:chararray,flight_number:int,tail_number:chararray,origin:chararray,destination:chararray,scheduled_departur
e:chararray,departure_time:chararray,departure_delay:double,taxi_out:double,wheels_off:chararray,scheduled_time:double,elapsed_time:double,air_time:double
,distance:double,wheels_on:chararray,taxi_in:double,scheduled_arrival:chararray,arrival_time:chararray,arrival_delay:double,diverted:int,cancelled:int,cancellati
on_reason:chararray,air_system_delay:double,security_delay:double,airline_delay:double,late_aircraft_delay:double,weather_delay:double);

valid = FILTER flights BY year IS NOT NULL AND arrival_delay IS NOT NULL;
grouped = GROUP valid BY month;
result = FOREACH grouped GENERATE group AS month, AVG(valid.arrival_delay) AS average_arrival_delay_minutes;
ordered = ORDER result BY month ASC;

STORE ordered INTO 'E:\BDAL\Final_Assignment\Flight_Data_Analysis\PigAnalysis\output\07_average_arrival_delay_by_month' USING PigStorage('|');
```

Command to run

```
pig -x local -f PigAnalysis\scripts\07_average_arrival_delay_by_month.pig
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>pig -x local -f PigAnalysis\scripts\07_average_arrival_delay_by_month.pig
Pig features: FILTER, GROUP_BY, AVG, ORDER_BY
Job completed successfully
```

```
E:\BDAL\Final_Assignment\Flight_Data_Analysis>hdfs dfs -cat /Final_Project/PIG/07_average_arrival_delay_by_month/part-r-00000
1|5.813582983416227
2|8.320500020850556
3|4.92067265685641
4|3.1631900611579318
5|4.485018615679651
6|9.601590351569554
7|6.4317747052785466
8|4.607372469025074
9|-0.7725709883956179
10|-0.780389663641748
11|1.100783576682592
12|6.092902747824754
```

Interpretation

This analysis allows arrival-delay performance to be examined from a monthly perspective. It can help identify months where flights experienced relatively higher or lower average arrival delays.

MapReduce and Pig Comparison

MapReduce and Apache Pig are both used in this project to process the flight dataset, but they provide different approaches to data analysis.

Feature	Hadoop MapReduce	Apache Pig
Programming method	Java programming	Pig Latin scripting
Abstraction level	Lower-level	Higher-level
Code complexity	More code required	Relatively simpler
Main operations	Mapper and Reducer	LOAD, FILTER, GROUP, JOIN, ORDER, etc.
Data processing	Key-value based	Data-flow based
Suitable for	Custom processing logic	Relational and analytical operations
Development effort	Higher	Lower
Dataset used	flights.csv	flights.csv and lookup data

MapReduce provides direct control over the Mapper and Reducer stages. This makes it suitable when custom processing logic is required. For example, the average arrival delay calculation is explicitly implemented through Mapper and Reducer classes.

Pig provides a higher-level approach. Operations such as grouping, counting, filtering, joining, sorting, and limiting can be performed using relatively short Pig Latin scripts.

In this project, both technologies successfully perform flight-data analysis. MapReduce demonstrates the underlying Hadoop processing model, while Pig provides a simpler method for performing similar analytical operations.

Results and Findings

The analysis of the 2015 flight dataset produced several important findings about airline activity, airport activity, flight volume, cancellations, routes, and delays.

The dataset contains **5,819,079 flight records**, providing a large collection of flight information for analysis.

The airline flight-count analysis shows that **WN** recorded the highest number of flights, with **1,261,855 flights**. This indicates that WN had the largest operational volume among the airlines shown in the results.

The airport analysis identified **ATL** as the busiest origin airport, with **346,836 departures**. Other highly active airports included ORD, DFW, DEN, and LAX.

The monthly analysis showed that flight activity changed throughout the year. Among the reported MapReduce results, **July had the highest monthly flight total with 520,718 flights**.

The cancellation analysis identified WN as having the highest number of cancelled flights among the reported results, with **16,043 cancellations**. Other airlines with relatively high cancellation counts included EV and MQ.

The delay analysis showed differences in airline delay performance. The MapReduce analysis identified **NK** as having the highest average arrival delay among the reported airlines, at **14.47 minutes**. The Pig analysis also identified NK as having the highest average departure delay, at approximately **15.94 minutes**.

The route analysis provided additional information about the structure of the U.S. flight network. The busiest directional route identified in the report was **SFO to LAX**, with **13,744 recorded flights**.

Overall, the results demonstrate how a large flight dataset can be processed using Hadoop MapReduce and Apache Pig to produce useful information. MapReduce provides detailed control over the processing stages, while Pig simplifies common analytical operations such as filtering, grouping, joining, sorting, and aggregation.

Conclusion

The full dataset contains 5,819,079 flight records. Southwest Airlines operated the most flights (1,261,855). ATL was the busiest origin airport with 346,836 departures. July had the highest monthly flight total at 520,718. The busiest directional route was SFO to LAX with 13,744 flights. The highest average departure delay was recorded by NK at approximately 15.94 minutes, while the highest average arrival delay was also NK at 14.47 minutes. These findings are derived directly from the executed MapReduce and Pig outputs.